Optical Character Recognition and Generation for Text CAPTCHA Images

Nguyen Hoang Nam; Danny Handoko; Yusen Luan; Qin Yan Er; Arshin Sikka

School of Computing, National University of Singapore

ABSTRACT

This mini-project aims to design and implement a good and simple OCR pipeline for a specific family of colored Text CAPTCHA with limited samples. The authors have tested multiple machine learning models, including ones implemented from scratch (CRNN + CTC, Segmentation + CNN (SegCNN)), ones trained from scratch (YOLO), and ones fine-tuned on our dataset (ViT). We also applied dataset-specific data preprocessing and post-processing techniques.

Our research shows that for this dataset, SegCNN with CRNN + CTC fallback is very competitive, boasting a 57% sequence accuracy and can be trained on 16GB of VRAM in 2 hours. We also explored image generation on this dataset with a fully convolutional Unconditional Latent Diffusion pipeline with KL+LPIPS VAE and Self-Attention U-Net and found that the model can generate with variety in both semantics and style.

Our contributions can be summarized as: providing a brief review of some models and pipelines' performance on the dataset of interest, along with a few design choices, implementations, and results of approaches tailored to this task on this dataset, and some results and engineering on data synthesis for this dataset.

METHODOLOGY (OCR)

1. Data Preprocessing

- Data was cleaned and preprocessed before training to improve model performance. We removed erroneous images (those with non alphanumeric symbols), relabeled for case-sensitivity (which did *not* help, as we got almost the same results with the original data, likely due to classes like i, I, l, 1, etc.) and ground-truth accuracy, and labeled 1750 images into YOLO format.
- Median Filtering (3x3 kernel) to remove line noise. We filtered black pixels (intensity < 50) to avoid filtering characters. We found this gave better results for our models. Adaptive Gaussian Thresholding was applied to binarize the images.

Fig 1. Generic Median Filter vs Custom Median Filter

2. Pipelines

CRNN + CTC

Model Architecture

- The CRNN integrates convolutional layers for spatial feature extraction, followed by recurrent layers for temporal decoding.

Fig 2. Model Architecture of CRNN model

Connectionist Temporal Classification (CTC)

- We applied the CTC algorithm as our loss function in training the CRNN. CTC loss enables the model to learn variable-length sequence outputs without explicit character-level alignment.

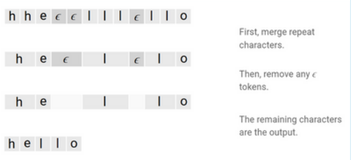


Fig 3. CTC Algorithm collapsing the predicted output (Hannun, 2017)

Training

- Training is performed using a custom train_step that directly computes CTC loss via ctc_batch_cost and applies gradient updates accordingly. Some additional data processing includes aspect-ratio-preserving resizing, label encoding, padding, and conservative augmentation (zoom-out scaling, horizontal stretching) applied on-the-fly each epoch. The training loop uses an 80/20 train–validation split, Adam optimization, LR scheduling, etc.

SegCNN

Model Architecture

We implemented a **hybrid pipeline** combining classical contour-based segmentation with a moderately deep CNN classifier, then combine character classifications back to the label. A segmentation cascade first produces glyph crops, which are then classified by a CNN for character recognition, and then sorted by x coordinate and recombined.

1. Segmentation Cascade

- The cascade stops when the number of contour blobs are similar to number of characters in the ground-truth. It starts with morphologically closing strokes with different types of dilation, namely none, vertical, very vertical, horizontal, and neighbors, then find contours of the image with Suzuki85 in O(N).
- If dilation fails, we find valleys in horizontal color frequency by summing over the number of white pixels in each column and check if the number of valleys correspond to number of characters in ground-truth.
- If all else fails, the watershed algorithm uses smart center finding, dilating and flooding. It's complexity is O(Na(N)). If this fails, segmentation fails.
- Valid crops are resized and padded to 64×64.
- Limitation: in general, handwritten CAPTCHAs have thin strokes, irregular spacing, and overlapping characters, making rule-based segmentation fundamentally unstable. But in the interest of time and resources, we assume it works well enough for our dataset.

2. CNN Classifier (ResNet-18)

Each crop is then classified using a **ResNet-18** initialized with ImageNet weights trained on this dataset.

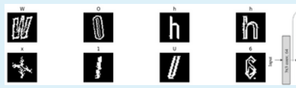


Fig 4. Glyph crops

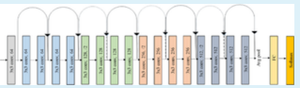


Fig 5. ResNet-18 Architecture

Other Models:

a. YOLO

- YOLOv12m acheives **70% character accuracy** and **38% sequence accuracy**. A notable setback is the data amount (1600), the quality, and difficulty of finding a IOU threshold for NMS to remove duplicate boxes but not valid ones (due to badly labelled data). Due to time, we could not do more hyperparameter tuning / data labeling, but this shows that a big bottleneck for this approach on this dataset is the **sparsity of manual well-labeled data.**

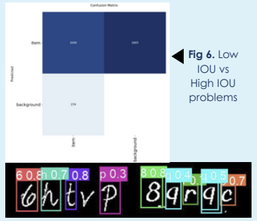


Fig 6. Low IOU vs High IOU problems

b. Vision Transformer (TrOCR)

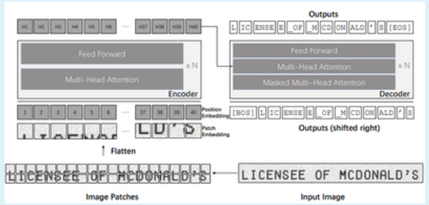


Fig 7a. Vision Transformer-based OCR architecture, which flatenned image patches embedding and fine-tuned using pretrained **"trocr-small-stage1" visual encoding-decoding.**[6]

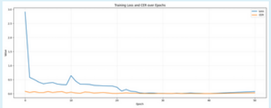


Fig 7b. Training Loss & Character Error Rate (CER) for 229 epochs

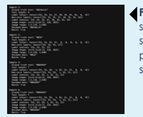


Fig 7c. Four samples to show GT vs predicted strings.

RESULTS & ANALYSIS (OCR)

The CRNN + CTC achieved a final character accuracy of **83%** and sequence accuracy of **48%**. Two most notable improvements were the use of Adam optimization, (char. accuracy **74% → 78%**, seq. accuracy **37% → 42%**), and the addition of noise filtering and binarization in the preprocessing step (char. accuracy **78% → 82%**, seq. accuracy **42% → 47%**). The addition of adaptive learning rate had greatly stabilized validation loss. Additionally, we trained again on all-lowercase and the accuracy did not change. The case sensitive relabeling performed in the beginning turned out to be not necessary / counter-productive.

Fig 8a. Training and Validation loss plot Over 100 epochs (with Adam)

Fig 8b. Training and Validation loss plot Over 100 epochs (without Adam)

The Segmentation CNN pipeline acheived **91% test dataset character accuracy and 56.2% sequence accuracy**. Most inaccuracies were either failed segmentation (5%), segmentation but with overlapping characters (20%), single character misclassification, which happens often with the groups {0, o, O}, {i, l, I, 1, |, !, 7}, {5, s, S}. Upon manual inspection, this is both a data labeling defect and an inevitability, as humans cannot perform with 100% accuracy on many of these.

```
--- Training Set Accuracy Report ---
Failed Segmentations:  0
Total Captchas: 7297
Correctly Predicted:   6820
Accuracy:              93.46%

--- Test Set Accuracy Report ---
Failed Segmentations:  104
Total Captchas: 2000
Correctly Predicted:   1124
Accuracy:              56.20%
```

Fig 9. Training accuracy reaching up to 93.5% on sequence

Fig 10a. ot8ski, preditced 6668wi

Fig 10b. rk5tu, preditced vkrt4

Fig 10c. v96l5, preditced v96i5

Fig 10d. jnovjd, preditced jn0vjd

When combined in the final pipeline, we achieved an extra 12 correct predictions, bringing our pipeline's grand total to a respectable **57% sequence accuracy and 90% character accuracy**, only **8% lower than a SOTA model while training way faster.** This means that, on average, **we can break into any site that uses this kind of CAPTCHA in 2 tries or less.**

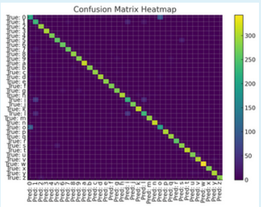


Fig 11. Confusion matrix showing the "trouble" groups during character classification

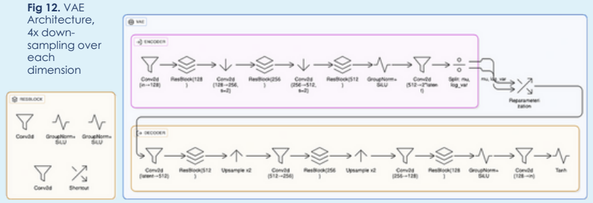
METHODOLOGY & RESULTS (GENERATIVE)

1. Data Preprocessing

Data loader and preprocessor resizes image height from 80 -> 64 and keep aspect ratio, then crop width to nearest multiple of 128. All data is normalized. Square images removed. Loader batches images by size. Data is generated at different ratios via fully convolutional architecture. Validation with 90/10 split.

2. Architecture & Results

Fig 12. VAE Architecture, 4x down-sampling over each dimension



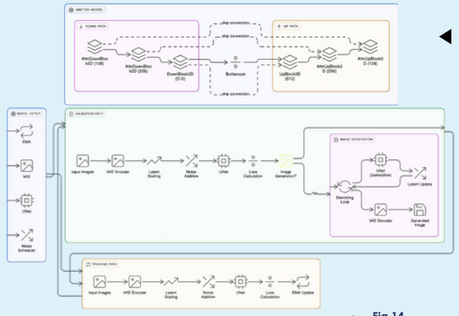


Fig 13. Diffusion pipeline and U-Net Architecture

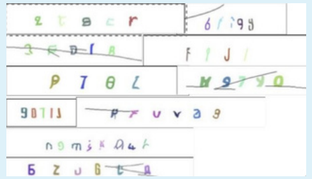


Fig 14. Samples

The generated images show both diversity in style (color, font, position, noise) and in the classes (number of characters, type of characters, size). This is the result of 150 – 250 epochs of diffusion training.

LIMITATIONS & FUTURE WORK

1. **Limited data and compute:** We were asked to use only provided training data. Therefore, it is possible to improve our baseline using conditional generative models to help generalize. This is an even bigger problem for the data-hungry LDM. We also skipped OBB for YOLO. We believe scaling models will bring better results.
2. **Similar characters:** Characters like 1, l and I are perceptually almost similar even to humans. For models to perform well, future work will need to integrate confusion-aware training with targeted augmentations or increase weightage of these classes in training. Character errors heavily snowball into sequence errors.
3. **Removal of color dimension:** We have tried segmentation methods on colored variants and found minimal to no improvement. We conclude that for these instances, more data and more complex models using Transformers are necessary for near-SOTA performance.

CONCLUSION

1. Certain simple types and families of Text CAPTCHAs can now be bypassed with relatively **light ML models and simple data engineering**, showing signs of unreliability of Text CAPTCHA as a Turing test in this age of ML/DL-based CV. More websites should use stronger alternatives like behavioral CAPTCHAs or 2FA.
2. Image synthesis for specific domains like Text CAPTCHA can be trained on **consumer-grade compute** with smaller models and simple architectures, while having **decent generalizability** -> effectiveness of LDM architecture for small to medium-scale tasks like generating image sketches for artists. We believe LDM will remain SOTA for the foreseeable future, even with *Flow Matching* on the horizon.
3. Home-brewed solutions trained on limited data and compute can be competitive on both accuracy and especially training speed with bigger models fine-tuned for specific downstream tasks with **good data engineering and design choices**. This is a good sign towards democratizing ML.

CONTRIBUTIONS

Everyone relabeled the data for case-sensitivity and removed noisy data.
1. Nguyen Hoang Nam designed the architecture, implemented, and benchmarked the VAE, U-Net, and Diffusion pipeline, labelled 470 YOLO images, trained and tuned YOLO, implemented, trained and tuned SegCNN, and compiled OCR literature.
2. Danny Handoko benchmarked our models against fine-tuned OCR models like ResNet-50 and ViT, helped Yusen Luan with CRNN CTC, and labelled 524 YOLO images.
3. Yusen Luan implemented, trained, and tuned CRNN CTC & labelled 200 YOLO images.
4. Qin Yan Er implemented data preprocessing pipeline & labelled 300 YOLO images.
5. Arshin Sikka implemented and trained SegCNN & labelled 300 YOLO images.

REFERENCES

1. Dayanand, Jeberson, W., & Jeberson, K. (2024). Unveiling CAPTCHA Vulnerabilities: Breaking CAPTCHA Using Deep Learning Techniques and Design and Development of Robust CAPTCHA Technique. *SSRG International Journal of Electrical and Electronics Engineering, 11*(6), 282–304.
2. Chen, J., Luo, X., Guo, Y., Zhang, Y., & Gong, D. (2017). A Survey on Breaking Technique of Text-Based CAPTCHA. *Security and Communication Networks, 2017*, Article 6898617.
3. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *arXiv preprint arXiv:2006.11239*.
4. Rombach, R., Blattmann, A., Ommer, B., & Esser, P. (2021). High-Resolution Image Synthesis with Latent Diffusion Models. *arXiv preprint arXiv:2112.10752*.
5. Awni Hannun (2017). Sequence ModelingWith CTC. Stanford University. https://doi.org/10.23915/distill.00008
6. Li, et.al. (2022). TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models. 10.48550/arXiv.2109.10282.

RESEARCH POSTER PRESENTATION DESIGN © 2015
www.PosterPresentations.com